

What is DSPy



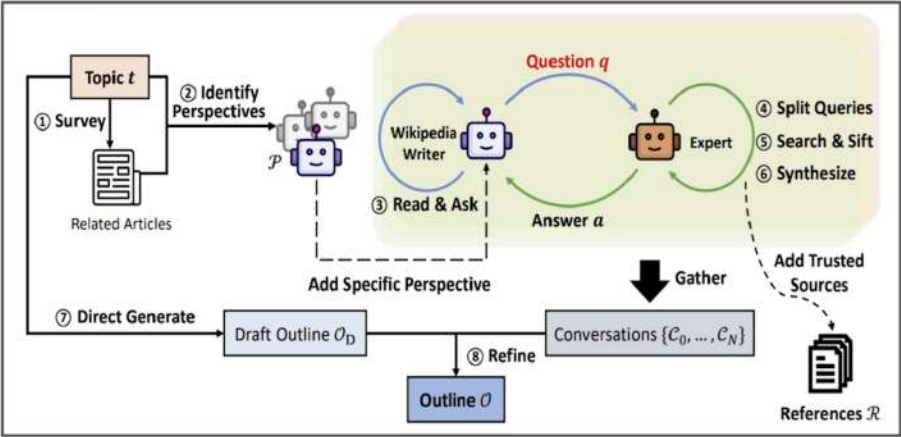
DSPy is a GenAI authoring framework, that simplifies the development of GenAI applications.

Use Cases

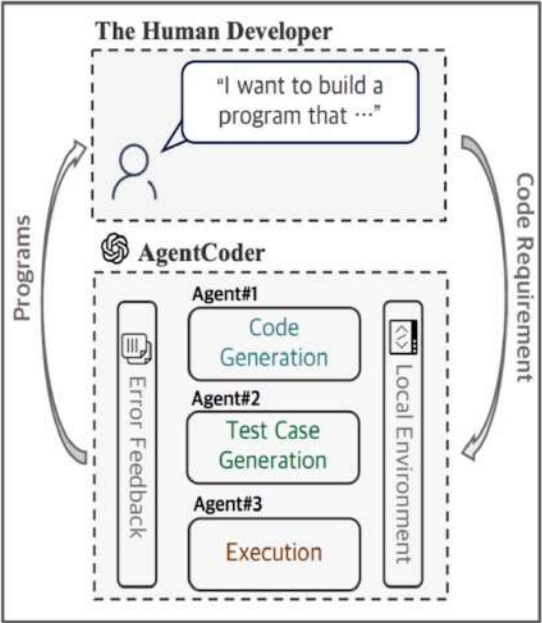
- Retrieval-augmented generation (RAG)
- Chain-of-thought prompting
- Structured output generation (like JSON)
- LLM pipelines for agents or tools

The Rise of Compound AI System

Question Answering AI system



Code Generator AI system



Prompt Engineering is Messy

✅ **Better prompts = better results** (But getting there isn't straightforward...)

❌ Prompt engineering is messy

- Involves trial-and-error with string tweaks
- Unclear what changes actually help

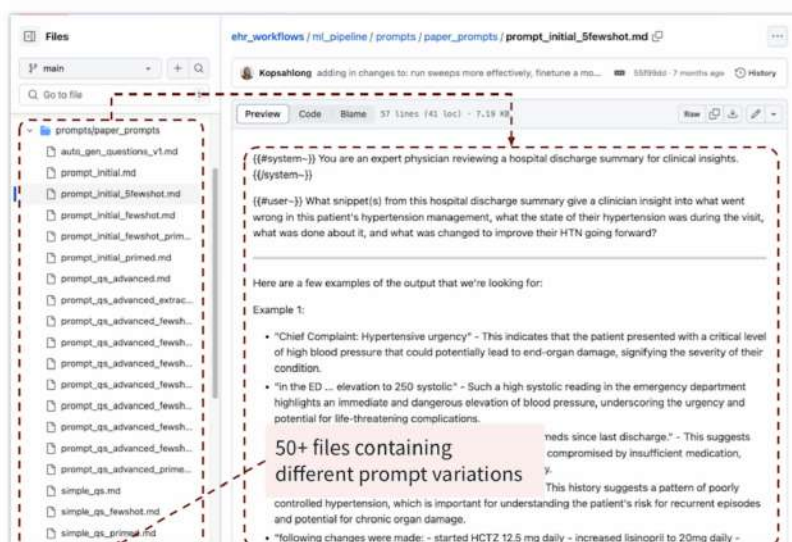
🔄 High iteration cost

- 50–100+ prompt variants per task
- Prompts can span **thousands of words**



📉 Model bias makes it worse

- Switching models = redoing all prompt tuning



Framework Can be Additional Troublemaker

⚙️ Frameworks simplify complex tasks

- Great for building agents, RAG pipelines, etc.

😞 Users often feel frustrated

- “The framework adds more pain than value.”

📖 Framework contract overhead

- Requires learning custom APIs and design patterns



🔒 Lock-in and migration pain

- Existing code becomes tightly coupled; Hard to switch frameworks later



This is the problem that DSPy aims solve.



DSPy

Programming — not prompting — Language Models

DSPy is a lightweight, flexible framework

- Simplifies interaction with language models (LMs)

Automatic program optimization

- Prompt optimization
- Fine-tuning LM weights with **DSPy Optimizer**

Ready for production

- Supports **streaming**, **async**, and more out of the box

Modular building blocks, no lock-in

- Use only what you need
- Easy to migrate **to** or **from** DSPy

What is Special About DSPy?

**LM-agnostic programing,
not LM-biased prompting**

Lesson 2

**Seamless Productization,
native features and MLflow
integration**

Lesson 3

**Automatic Program
Optimization**

Lesson 4

Trusted by Industry

A Few Company Use Cases

Name	Use Cases
JetBlue	Multiple chatbot use cases. Blog
Replit	Synthesize diffs using code LLMs using a DSPy pipeline. Blog
Databricks	Research, products, and customer solutions around LM Judges, RAG, classification, and other applications. Blog , Blog II
Sephora	Undisclosed agent usecases; perspectives shared in DAIS Session.
Zoro UK	E-commerce applications around structured shopping. Portkey Session
VMware	RAG and other prompt optimization applications. Interview in The Register . Business Insider .
Haize Labs	Automated red-teaming for LLMs. Blog
Plastic Labs	R&D pipelines for Honcho. Blog
PingCAP	Building a knowledge graph. Article
Salomatic	Enriching medical reports using DSPy. Blog

Check it out:

<https://dspy.ai/community/use-cases>

Lab Materials

- Build a simple sentiment analysis program with DSPy built-in modules.
- A custom agent with DSPy by building a “Name the Celebrity” game.



DSPy Backbone – Signature and Module

The backbone of DSPy programming consists of two classes:

- 1) `dspy.Signature`
 - 2) `dspy.Module`.
- **dspy.Signature:** define the input/output contract of DSPy module

```
classify = dspy.Predict("sentence -> sentiment: int, language: str")
```



- **dspy.Module:** define the logic of interacting with LM. Takes user input and signature.

```
cot = dspy.ChainOfThought("question->answer")  
print(cot(question="what's the capital of France?"))
```

Signature

Signature (**dspy.Signature**) defines the input and output of LLM interaction.

Think of LLM as a function, signature defines the args and return value of the function.

- What I can give you (**dspy.InputField()**)
- What you should give me (**dspy.OutputField()**)
 - Type (**pydantic** format)
 - Extra information about the field via **desc** keyword

Two types of signatures:

- Class-based
- String-based

Class-based Signature

```
class Translation(dspy.Signature):  
    """Translate English sentence into Japanese."""  
  
    english_text: str = dspy.InputField()  
    japanese_text: str = dspy.OutputField()  
    hiragana: str = dspy.OutputField()  
    grammer_involved: list[str] = dspy.OutputField()  
    alternatives: list[tuple[str, str]] = dspy.OutputField(  
        desc="Alternatives and explanation"  
    )  
    difficulty: Literal["N1", "N2", "N3", "N4", "N5"] = dspy.OutputField(  
        desc="JLPT level"  
    )
```



Instruction



Extra information for the field



Field name



Type information



Input or output field

String-based Signature

```
classify = dsp.Predict(sentence -> sentiment: int, language: str)
```

Left side of "->" are the input fields, and right side are the output fields, separated by ","

Built-in Modules

 **Module (`dspy.Module`) is the core unit of a DSPy program**

- Encapsulates logic for interacting with LMs

 **Simplest module: `dspy.Predict`**

- Formats user input
- Parses LM output based on a signature

 **Modules have configurable attributes**

- `demos`: Few-shot examples
- `lm`: Specifies which language model to use

  **Modules are composable and customizable**

- Can contain child modules
- Add custom logic for advanced workflows

Built-in Modules

Inside DSPy we provide a list of built-in modules to make it easy for users to get started:

- **dspy.Predict**: Simple LM interaction. The building block for complex modules
- **dspy.ChainOfThought**: ask LM to include reasoning along with the answer
- **dspy.ReAct**: the common abstraction of continuous tool calling
- **dspy.ProgramOfThought**: a special tool calling module, inside it the tool is code
- **dspy.Refine**: users can set a reward function + threshold, if threshold is unmet we issue retries.

Built-in Modules

```
cot = dsp.ChainOfThought("question->answer")  
print(cot(question="what's the capital of France?"))
```

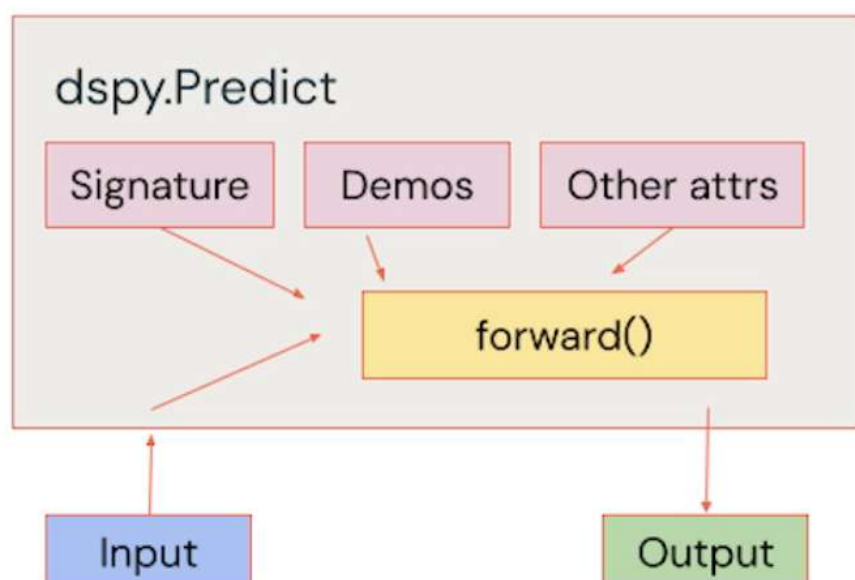
Built-in modules: <https://dsp.ai/api/modules/Module/>

Custom Modules

```
class MyProgram(dspy.Module):  
    def __init__(self, **kwargs):  
        super().__init__()  
  
        self.module1 = dspy.ChainOfThought(MySignature1)  
        self.module2 = dspy.ChainOfThought(MySignature2)  
  
    def forward(self, inputs, **kwargs):  
        x = preprocess(x)  
        x = self.module1(inputs)  
        x = midprocess(x)  
        x = self.module2(x)  
        return postprocess(x)
```

Define your forward logic in `forward` method, similar to PyTorch

Behind the Scene - Adapter



At a high level, **`dspy.Predict`** Composes a multi-turn prompt using:

-  **User input** — formatted by the **signature** and **adapter**
-  **Demos** — few-shot examples to guide the LM

Parses the LLM response

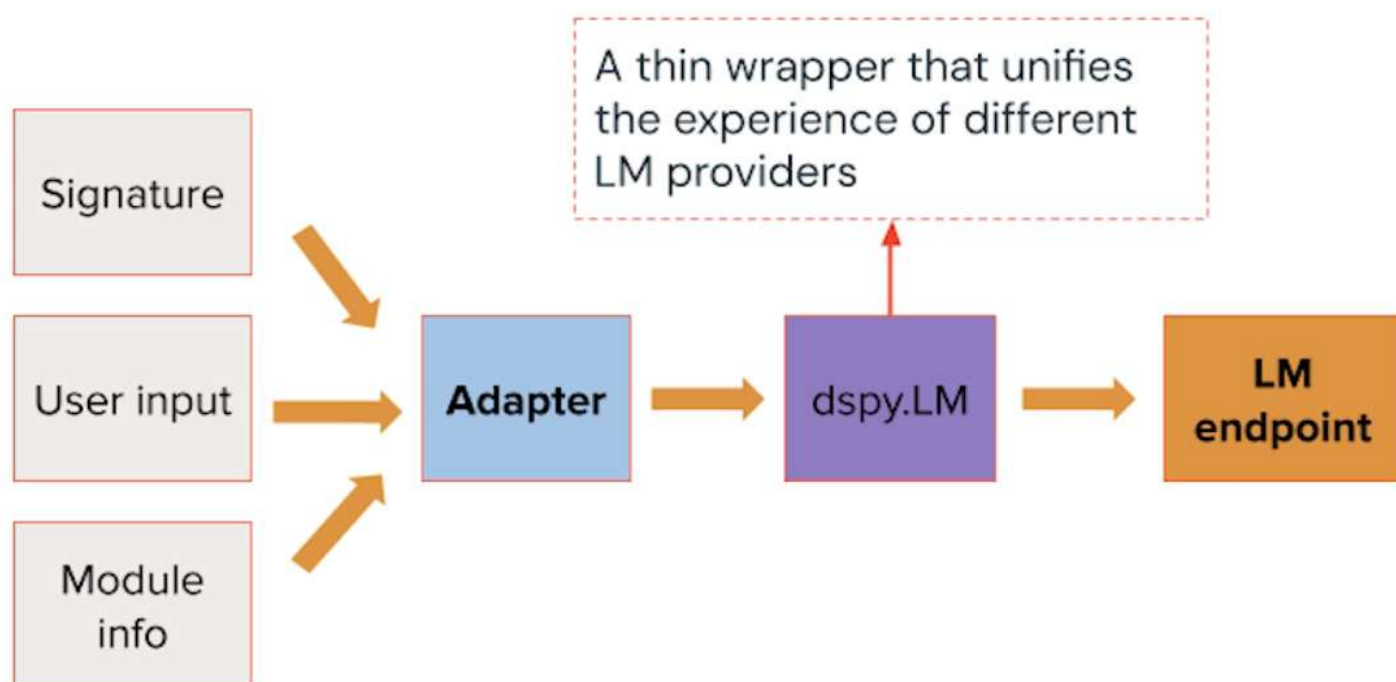
- Uses the same signature and adapter for structured output

Returns a `dspy.Prediction`

- A dict-like object containing the extracted output fields

Behind the Scene - Adapter (Input flow)

The adapter takes in signature, user input and module input (prebuilt few-shot examples), and transforms it into the familiar multi-turn messages with different roles.



Into the Adapter Input Flow

System message:

Your input fields are:

1. `question` (str)

Field Description

Source: Signature -> fields name

Your output fields are:

1. `reasoning` (str)

2. `answer` (str)

All interactions will be structured in the following way, with the appropriate values filled in.

[[## question ##]]

{question}

Input/output format description

Source: Signature -> fields name + info

[[## reasoning ##]]

{reasoning}

[[## answer ##]]

{answer}

[[## completed ##]]

In adhering to this structure, your objective is:

Given the fields `question`, produce the fields `answer`.

User message:

User message formatted as required

Source: Signature, user input

[[## question ##]]

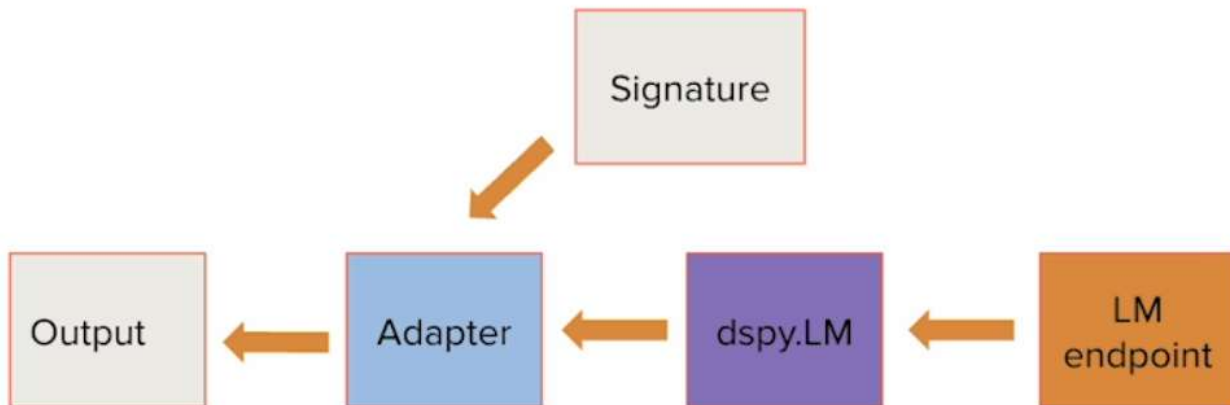
What is the meaning of life?

Respond with the corresponding output fields, starting with the field

[[## reasoning ##]], then [[## answer ##]], and then ending

with the marker for [[## completed ##]].

Behind the Scene - Adapter (Output flow)



The adapter takes in signature and LM response to parse the response into the user-specified fields.

Into the Adapter Output Flow

User message:

```
[[ ## question ## ]]
```

What *is* the meaning of life?

Respond *with* the corresponding output fields, starting *with* the field `[[ ## reasoning ## ]]', then `[[## answer ##]]', and then ending *with* the marker *for* `[[## completed ##]]'.

Response:

```
[[ ## reasoning ## ]]
```

The meaning of life *is* a philosophical question...

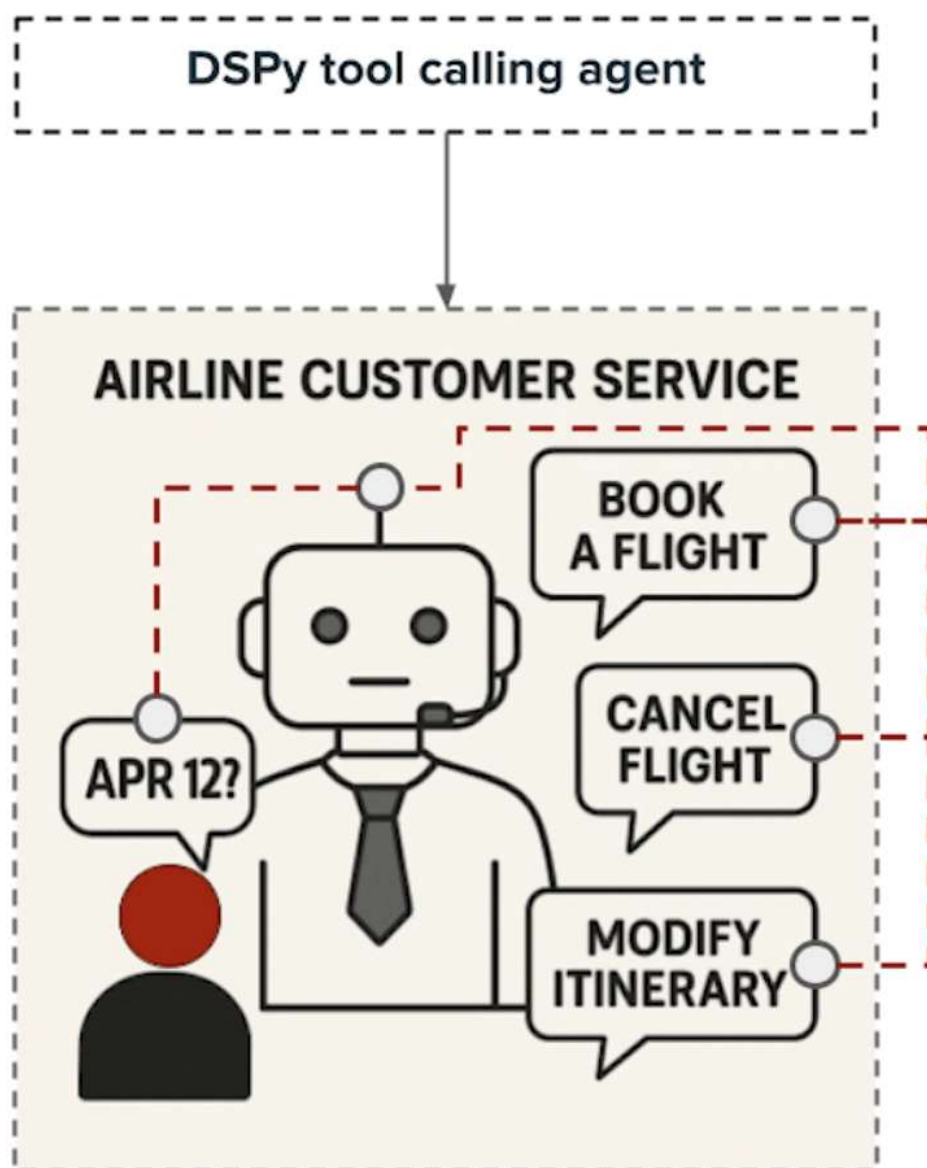
```
[[ ## answer ## ]]
```

The meaning of life *is* a subjective concept that varies...

```
[[ ## completed ## ]]
```

Prediction(
 reasoning="The meaning of life is a philosophical question...",
 answer="The meaning of life is a subjective concept that
varies..."

Lab Materials



What is tracing, and why tracing?

- **Tracing:** Records **inputs and outputs** of intermediate steps and captures **hierarchical call stack**

- **GenAI apps are complex:** Internal logic is often opaque

- **Without tracing: One failed LM** call among many can **break the whole program**

Tracing improves:

- ✓ Interpretability

- ✓ Debugging

What is MLflow

Open source MLOps framework

- Supports **ML and GenAI app development**
- Covers the **full lifecycle**: development, tracking, deployment

Focus on reproducibility and traceability

- Manage every experiment, run, and model version with ease

Open source server + client

- Fully open and extensible

Get Started with Tracing DSPy

```
import mlflow
```

```
mlflow.dspy.autolog()
```



This is all you need!

```
class MyProgram(dspy.Module):
```

```
    def __init__(self, **kwargs):
```

```
        super().__init__()
```

```
        self.module1 = dspy.ChainOfThought(MySignature1)
```

```
        self.module2 = dspy.ChainOfThought(MySignature2)
```

```
    def forward(self, inputs, **kwargs):
```

```
        x = preprocess(x)
```

```
        x = self.module1(inputs)
```

```
        x = midprocess(x)
```

```
        x = self.module2(x)
```

```
        return postprocess(x)
```

```
my_program = MyProgram()
```

```
my_program(inputs="what's the meaning of life")
```

MLflow Trace UI

The screenshot displays the MLflow 2.21.3 interface. The top navigation bar includes 'Experiments', 'Models', and 'Prompts'. On the left, the 'Experiments' sidebar shows a search bar and a list with 'Default' and 'dspy_workshop' (selected). The main area is titled 'dspy_workshop' and includes links for 'Provide Feedback' and 'Add Description'. Below the title are tabs for 'Runs', 'Evaluation', 'Experimental' (active), and 'Traces'. A search bar for traces and a 'Columns' dropdown are present. A table lists the traces with columns: Request ID, Time created, Status, Request, Response, and Run name. One trace is shown with a status of 'OK'.

Request ID	Time created	Status	Request	Response	Run name
858dea58bb96480eb82a0...	5 days ago	OK	{"user_request... help me book a flight from SFO to JFK on 09/01/2025, my name is Adam"}	> {"trajector... {"thought_... "I need to fetch flight information for Adam from SFO to	

Traces are linked to MLflow experiments.

MLflow DSPy Trace Structure

The following DSPy components are traced:

- Module
- Adapter
- LM
- Tool

ReAct.forward

Search
Task name

	Inputs / Outputs	Attributes	Events
ReAct.forward 13.48s	Inputs <pre>user_request</pre> please help me book a flight from SFO to JFK on 09/01/2025, my name is Adam Outputs <pre>trajectory</pre> <pre>{ "thought_0": "I need to fetch flight information for Adam from SFO to JFK", "tool_name_0": "fetch_flight_info", "tool_args_0": { "from": "SFO", "to": "JFK", "date": "09/01/2025", "name": "Adam" } }</pre> reasoning		
Predict.forward_1 2.20s	<> ChatAdapter.format_1 0.01s		
LM.__call__ 2.19s	LM.__call__ 2.19s		
ChatAdapter.parse_1 0.01s			
Tool.fetch_flight_info 0.01s			
Predict.forward_2 2.85s	<> ChatAdapter.format_2 0.01s		
LM.__call__ 2.85s	LM.__call__ 2.85s		
ChatAdapter.parse_2 0.01s			
Tool.pick_flight_1 0.01s			
.forward_3 2.58s			

Show timeline

Use MLflow Trace to Interpret Prompt



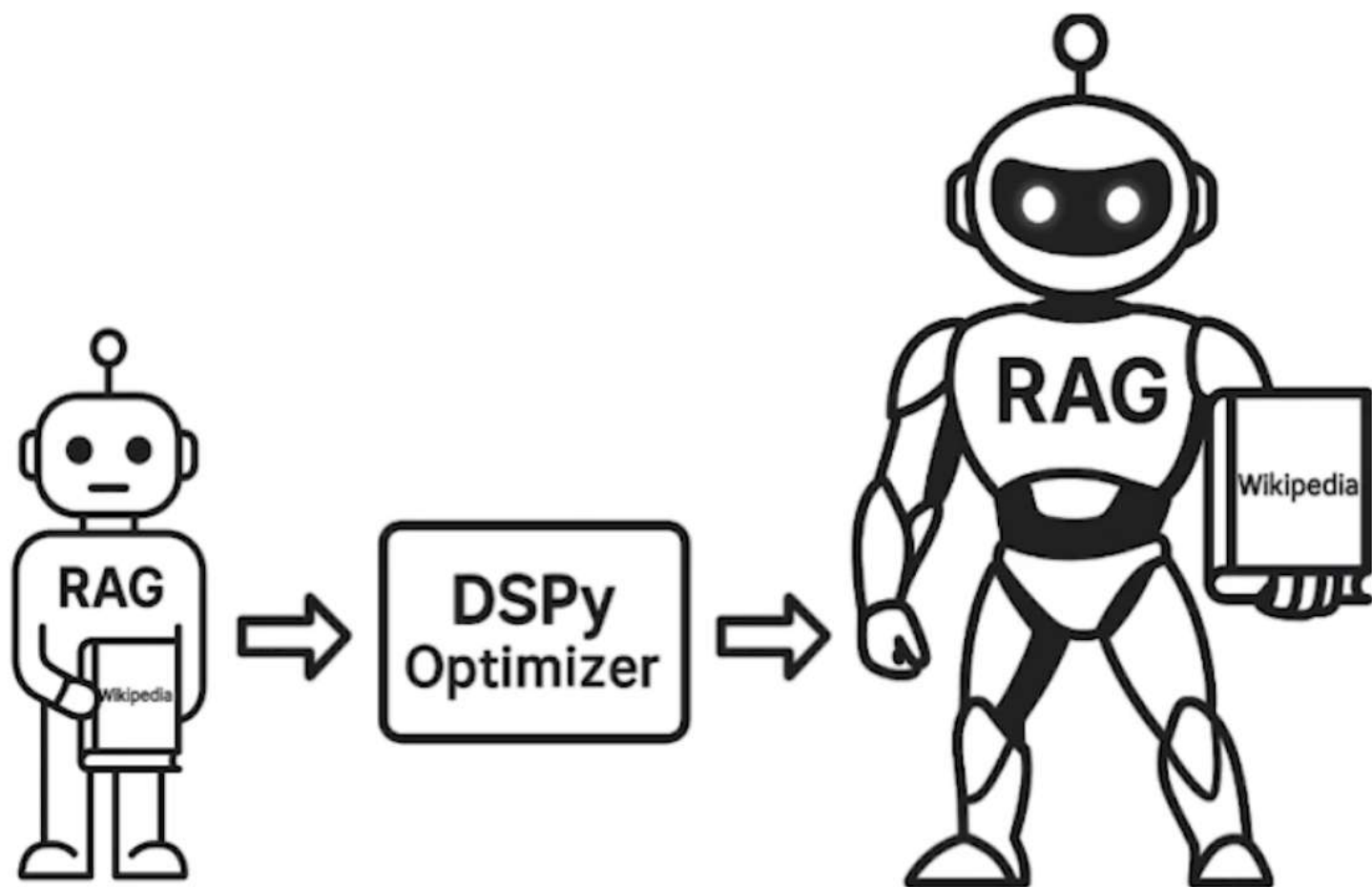
Let's see all of this in the lab

Use Managed MLflow Service

Databricks lighthouse: signup.databricks.com

Lab Materials

- Use **DSPy Optimizer**: Automatically improve the quality of a **RAG agent**.
- The data source is **Wikipedia**.



What does optimization mean?

1) Prompt template



- Task description
- Input/output description
- Language specs

2) Few-shot examples



- High-quality examples lead to good response quality
- Tradeoff between cost and quality

3) LM weights



- SFT to let the model learn more about the task
- Preference finetuning (GRPO and else) to have the model align the text

How to Use DSPy Optimizer

```
def search_wikipedia(query: str) -> list[str]:
    results = dspy.ColBERTv2(
        url="http://20.102.90.50:2017/wiki17_abstracts",
    )(query, k=3)
    return [x["text"] for x in results]

trainset = [
    x.with_inputs("question")
    for x in HotPotQA(
        train_seed=2024,
        train_size=500,
    ).train
]

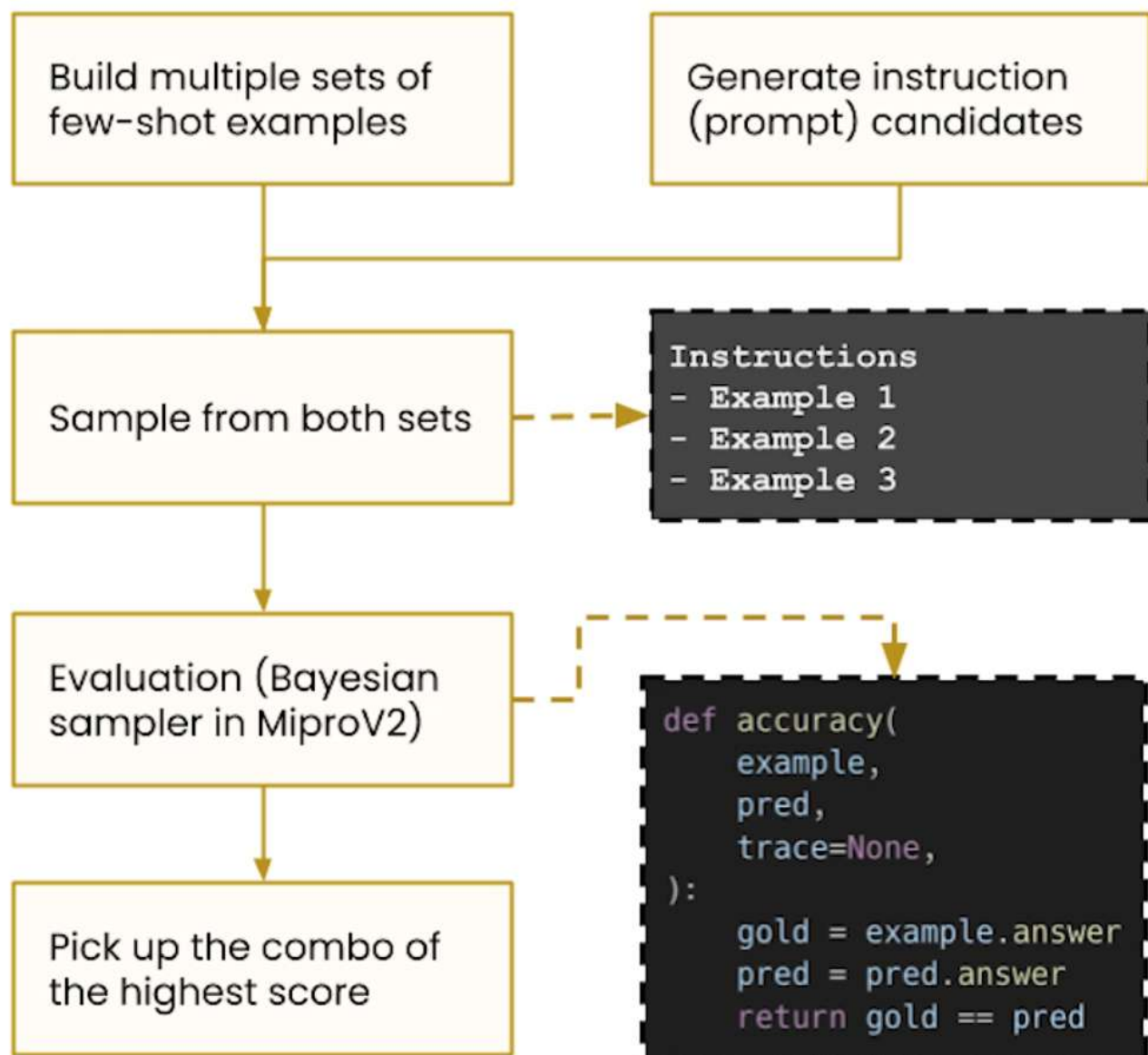
react = dspy.ReAct(
    "question -> answer",
    tools=[search_wikipedia],
)

tp = dspy.MIPROv2(
    metric=dspy.evaluate.answer_exact_match,
    auto="light",
    num_threads=24,
)

optimized_react = tp.compile(react, trainset=trainset)
```



Prompt Optimization



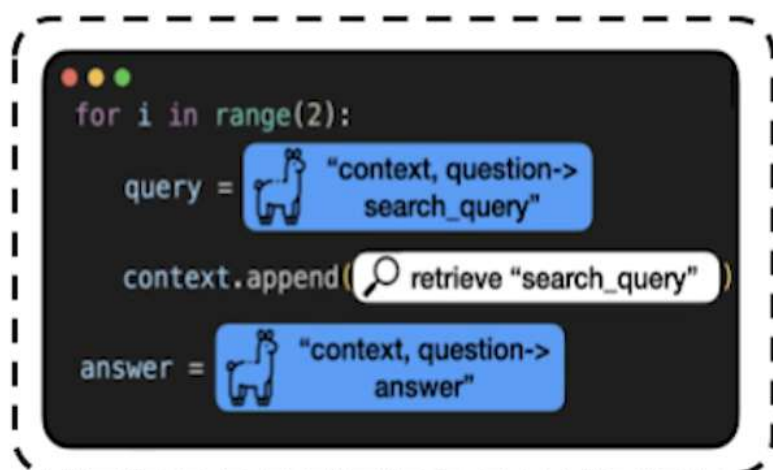
Build Few-shot Examples

Bootstrapping

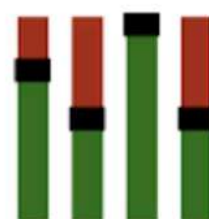
Training Input



LM Program:

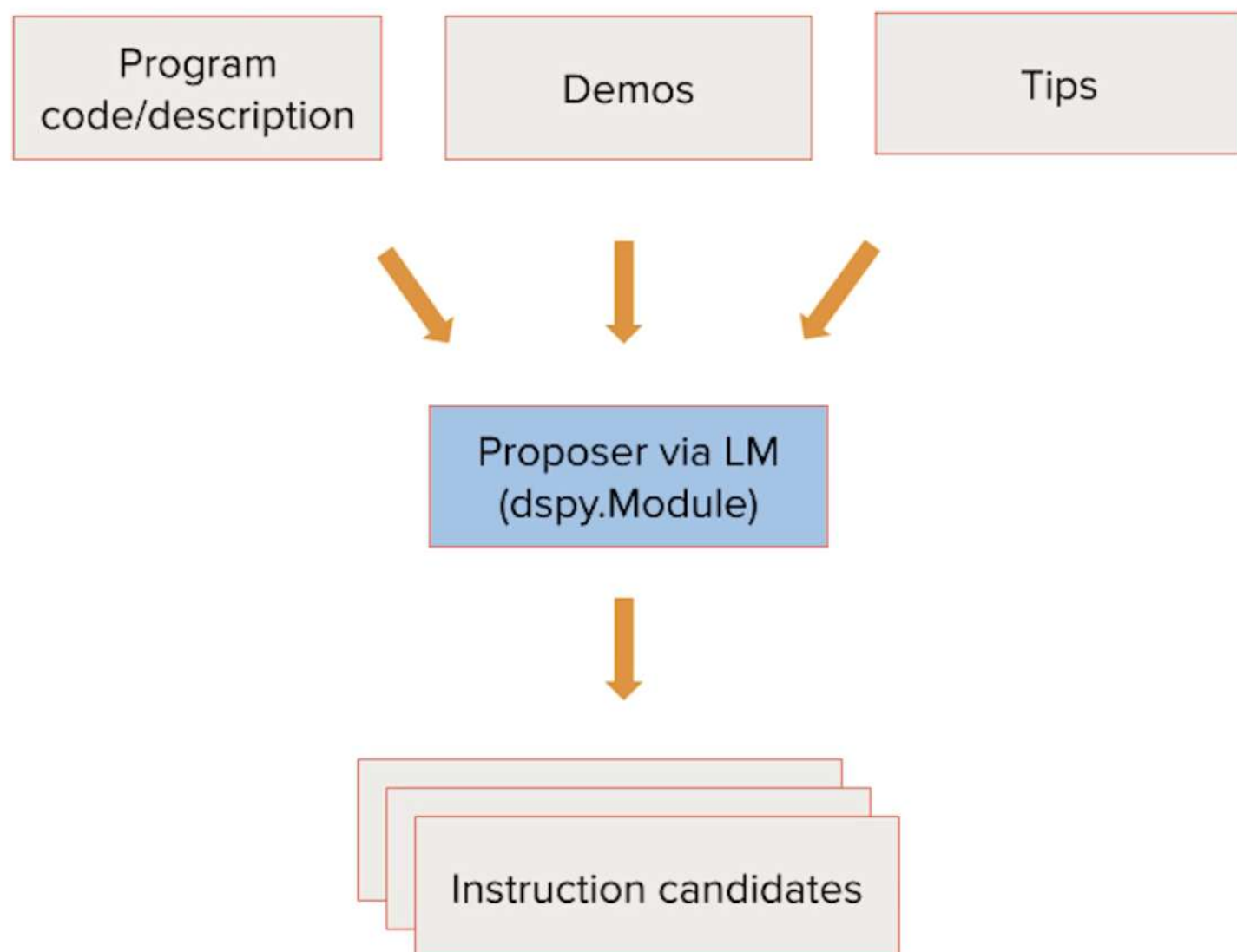


Metric



Good Trace!

Build Instruction Candidates



Optimizer Performance

Optimizer	ScoNe			HotPotQA			HoVer			HotPotQA Cond.			Iris		Iris-Typo		Heart Disease	
	Train	Dev	Test	Train	Dev	Test	Train	Dev	Test	Train	Dev	Test	Train	Test	Train	Test	Train	Test
<i>Instructions only (0-shot)</i>																		
N/A	57.0	56.2	69.1	35.4	31.8	36.1	30.2	30.8	25.3	13.8	10.5	6	46.4	40.9	34.7	32	23.3	26.8
Module-Level OPRO -G	70.0	67.4	76.1	36.0	31.7	36.0	30.0	30.0	25.7	-	-	-	-	-	-	-	-	-
Module-Level OPRO	69.1	67.6	73.5	41.9	36.2	39.0	37.1	38.6	32.5	-	-	-	-	-	-	-	-	-
0-Shot MIPRO	66.3	65.2	71.5	40.2	34.2	36.8	37.7	38.4	33.1	22.6	20.3	14.6	40.8	36.4	56.8	56.7	26.8	25.8
0-Shot MIPRO++	69.0	66.9	75.7	41.5	36.2	39.3	37.1	37.3	32.6	-	-	-	-	-	-	-	-	-
<i>Demonstrations only (Few-shot)</i>																		
Bootstrap RS	74.9	69.6	75.4	48.6	44.0	45.8	42.0	42.0	37.2	16.4	15.0	10.4	95.2	94.1	58.9	58.7	78.4	79.2
Bayesian Bootstrap	75.4	67.4	77.4	49.2	44.8	46.2	44.6	44.7	37.6	-	-	-	-	-	-	-	-	-
<i>Both (Few-shot)</i>																		
MIPRO	74.6	69.8	79.4	49.0	43.9	46.4	44.7	46.7	39.0	28.4	28.1	23.3	98.4	88.6	69.1	68.7	75.2	74.2

Paper: "Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs"

- <https://arxiv.org/pdf/2406.11695>

Track DSPy Optimization via MLflow



```
mlflow.dspy.autolog(  
    log_evals=True,  
    log_compiles=True,  
    log_traces_from_compile=True,  
)
```

MLflow tracks each evaluate call during optimization process.



Let's see all of this in the lab